

【计算最接近的数】

题目描述

给定一个数组X和正整数K，请找出使表达式：

$$X[i] - X[i + 1] - \dots - X[i + K - 1]$$

结果最接近于数组 **中位数** 的下标 i ，如果有多个 i 满足条件，请返回最大的 i 。

其中，数组中位数：长度为N的数组，按照元素的值大小 **升序排列** 后，下标为 $N/2$ 元素的值

输入描述

无

输出描述

无

备注

1. 数组X的元素均为正整数
2. X的长度 n 取值范围： $2 \leq n \leq 1000$
3. K大于0且小于数组的大小
4. i 的取值范围： $0 \leq i < 1000$
5. 题目的排序数组X[N]的中位数是X[N/2]

用例

输入	[50,50,2,3],2
输出	1
说明	1. 中位数为50：[50,50,2,3]升序排序后变成 [2,3,50,50]，中位数为下标4/2=2的元素50 2. 计算结果为1：X [50,50,2,3] 根据题目计算X[i] - ... - X[i + k - 1] 得出三个数0 (X[0] - X[1] = 50 - 50)、48 (X[1] - X[2] = 50 - 2) 和 -1 (X[2]-X[3] = 2 - 3)，其中48最接近50，因此返回下标1。

题目解析

本题应该是采用核心代码模式，非ACM模式，因此不需要我们处理输入输出。

下面代码仍然以ACM模式实现，但是会将输入输出处理 和 核心代码 分离。考试时，只需要写出核心代码即可。

本题可以使用滑动窗口解题。

本题其实就是要遍历所有长度为k的滑窗，滑窗内部元素需要按照表达式

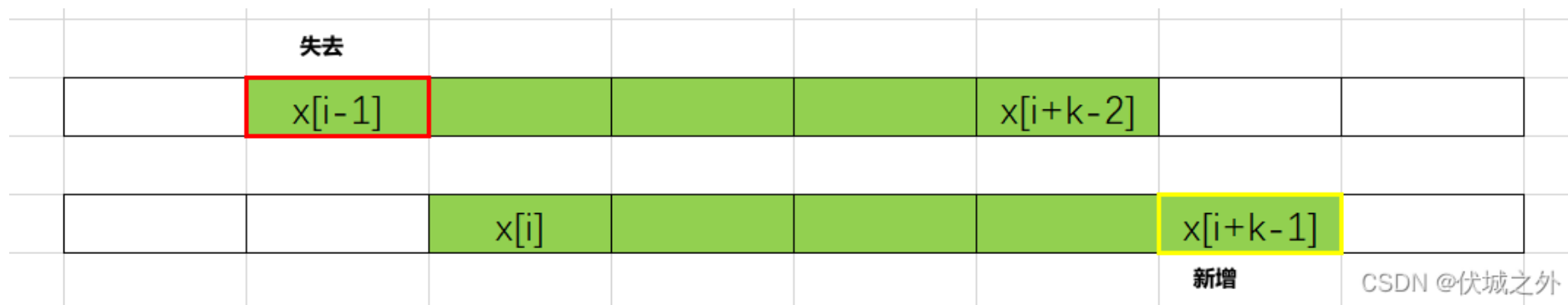
X[i] - X[i + 1] - ... - X[i + K - 1]

来求得该滑窗对应得 表达式计算 结果window。

然后，求解window和数组x中位数mid的差距，差距绝对值越小，则说明二者越接近

结果最接近于数组中位数的下标 i，如果有多个 i 满足条件，请返回最大的 i。

本题的难点在于滑窗右移时，新、旧滑窗的状态转移。



假设旧滑窗的表达式计算结果为window，那么新滑窗的表达式计算结果应该是：

$$\text{window} - x[i-1] + x[i] * 2 - x[i+k-1]$$

其中 $x[i] * 2$ 的原因是，新滑窗中 $x[i]$ 是正的，而旧滑窗中 $x[i]$ 是负的，为了将 $-x[i]$ 变为 $+x[i]$ ，则需要为 $-x[i] + x[i] * 2$

JS算法源码

```
1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5    input: process.stdin,
6    output: process.stdout,
7  });
8
9  rl.on("line", (line) => {
10    const i = line.lastIndexOf(",");
11
12    const x = line
13      .slice(1, i - 1)
14      .split(",")
```

运行

```
15     .map(Number);16
17     const k = parseInt(line.slice(i + 1));
18
19     console.log(getResult(x, k));
20 });
21
22 function getResult(x, k) {
23     const n = x.length;
24
25     const midIdx = Math.floor(n / 2);
26     // x数组的中位数
27     const mid = [...x].sort((a, b) => a - b)[midIdx];
28
29     // 初始化滑窗 $0 \sim k-1$ , window为滑窗内部元素的表达式计算结果
30     let window = x[0];
31     for (let i = 1; i < k; i++) {
32         window -= x[i];
33     }
34
35     // window和中位数的差距
36     let minDiff = Math.abs(mid - window);
37     // window滑窗起始索引
38     let idx = 0;
39
40     // 滑窗右移
41     for (let i = 1; i <= n - k; i++) {
42         // 右移一格后, 新滑窗的表达式计算结果
43         window += -x[i - 1] + 2 * x[i] - x[i + k - 1];
44
45         // 新滑窗window值和中位数的差距
46         const diff = Math.abs(mid - window);
47
48         // 结果最接近于数组中位数的下标 i , 如果有多个 i 满足条件, 请返回最大的 i
49         if (diff <= minDiff) {
50             minDiff = diff;
51             idx = i;
```

```
52 |     }53 | }
54 |
55 |     return idx;
56 | }
```

Java算法源码

```
1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |
8 |         String line = sc.nextLine();
9 |
10 |        int i = line.lastIndexOf(",");
11 |
12 |        int[] x =
13 |            Arrays.stream(line.substring(1, i - 1).split(",")).mapToInt(Integer::parseInt).toArray();
14 |        int k = Integer.parseInt(line.substring(i + 1));
15 |
16 |        System.out.println(getResult(x, k));
17 |    }
18 |
19 |    public static int getResult(int[] x, int k) {
20 |        int n = x.length;
21 |
22 |        // x数组的中位数
23 |        int mid = Arrays.stream(x).sorted().toArray()[n / 2];
24 |
25 |        // 初始化滑窗0~k-1, window为滑窗内部元素的表达式计算结果
```

```

26     int window = x[0];27     for (int i = 1; i < k; i++) {
28         window -= x[i];
29     }
30
31     // window和中位数的差距
32     int minDiff = Math.abs(mid - window);
33     // window滑窗起始索引
34     int idx = 0;
35
36     // 滑窗右移
37     for (int i = 1; i <= n - k; i++) {
38         // 右移一格后，新滑窗的表达式计算结果
39         window += -x[i - 1] + 2 * x[i] - x[i + k - 1];
40
41         // 新滑窗window值和中位数的差距
42         int diff = Math.abs(mid - window);
43
44         // 结果最接近于数组中位数的下标 i ，如果有多个 i 满足条件，请返回最大的 i
45         if (diff <= minDiff) {
46             minDiff = diff;
47             idx = i;
48         }
49     }
50
51     return idx;
52 }
53 }

```

Python算法源码

```

1 # 输入获取
2 tmp = input()
3
4 i = tmp.rfind(",")

```

```
5 |
6 | x = list(map(int, tmp[1:i-1].split(",")))
7 | k = int(tmp[i+1:])
8 |
9 |
10 | # 核心代码
11 | def getResult():
12 |     n = len(x)
13 |
14 |     # x数组的中位数
15 |     mid = sorted(x)[n // 2]
16 |
17 |     # 初始化滑窗0~k-1, window为滑窗内部元素的表达式计算结果
18 |     window = x[0]
19 |     for j in range(1, k):
20 |         window -= x[j]
21 |
22 |     # window和中位数的差距
23 |     minDiff = abs(mid - window)
24 |     # window滑窗起始索引
25 |     idx = 0
26 |
27 |     # 滑窗右移
28 |     for i in range(1, n-k+1):
29 |         # 右移一格后, 新滑窗的表达式计算结果
30 |         window += -x[i-1] + 2 * x[i] - x[i + k - 1]
31 |
32 |         # 新滑窗window值和中位数的差距
33 |         diff = abs(mid - window)
34 |
35 |         # 结果最接近于数组中位数的下标 i , 如果有多个 i 满足条件, 请返回最大的 i
36 |         if diff <= minDiff:
37 |             minDiff = diff
38 |             idx = i
39 |
40 |     return idx
41 |
```

```
42 |  
    43 | # 算法调用  
44 | print(getResult())
```

C算法源码

```
1  #include <stdio.h>  
2  #include <string.h>  
3  #include <stdlib.h>  
4  
5  #define MAX_SIZE 1000  
6  
7  int cmp(const void *a, const void *b) {  
8      return (*(int *) a) - (*(int *) b);  
9  }  
10  
11 int getResult(const int x[], int n, int k) {  
12     int x_cp[n];  
13     for(int i=0; i<n; i++) x_cp[i] = x[i];  
14  
15     // 这里排序的是x_cp数组, 不能改变原数组x  
16     qsort(x_cp, n, sizeof(int), cmp);  
17  
18     // x数组的中位数  
19     int mid = x_cp[n / 2];  
20  
21     // 初始化滑窗0~k-1, window为滑窗内部元素的表达式计算结果  
22     int window = x[0];  
23     for (int i = 1; i < k; i++) {  
24         window -= x[i];  
25     }  
26  
27     // window和中位数的差距  
28     int minDiff = abs(mid - window);
```

```

29 |
30 | // window滑窗起始索引
31 | int idx = 0;
32 |
33 | // 滑窗右移
34 | for (int i = 1; i <= n - k; i++) {
35 |     // 右移一格后，新滑窗的表达式计算结果
36 |     window += -x[i - 1] + 2 * x[i] - x[i + k - 1];
37 |
38 |     // 新滑窗window值和中位数的差距
39 |     int diff = abs(mid - window);
40 |
41 |     // 结果最接近于数组中位数的下标 i，如果有多个 i 满足条件，请返回最大的 i
42 |     if (diff <= minDiff) {
43 |         minDiff = diff;
44 |         idx = i;
45 |     }
46 | }
47 |
48 | return idx;
49 | }
50 |
51 | int main() {
52 |     char s[100000];
53 |     scanf("%s", s);
54 |
55 |     char *p = strrchr(s, ',');
56 |     *p = '\0';
57 |
58 |     int nums[MAX_SIZE];
59 |     int nums_size = 0;
60 |
61 |     char *token = strtok(s + 1, ",");
62 |     while (token != NULL) {
63 |         nums[nums_size++] = atoi(token);
64 |         token = strtok(NULL, ",");
65 |     }

```

```

66 |         int k = atoi(p + 1);
67 |
68 |
69 |         printf("%d\n", getResult(nums, nums_size, k));
70 |
71 |         return 0;
72 |     }
73 |

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | int solution(vector<int> &x, int k) {
6 |     vector<int> x_cp(x);
7 |     sort(x_cp.begin(), x_cp.end());
8 |
9 |     // x数组的中位数
10 |    int mid = x_cp[x.size() / 2];
11 |
12 |    // 初始化滑窗0~k-1, window为滑窗内部元素的表达式计算结果
13 |    int window = x[0];
14 |    for (int i = 1; i < k; i++) {
15 |        window -= x[i];
16 |    }
17 |
18 |    // window和中位数的差距
19 |    int minDiff = abs(mid - window);
20 |
21 |    // window滑窗起始索引
22 |    int idx = 0;
23 |
24 |    // 滑窗右移
25 |    for (int i = 1; i <= x.size() - k; i++) {

```

```

26 |         // 右移一格后，新滑窗的表达式计算结果
27 |         window += -x[i - 1] + 2 * x[i] - x[i + k - 1];
28 |
29 |         // 新滑窗window值和中位数的差距
30 |         int diff = abs(mid - window);
31 |
32 |         // 结果最接近于数组中位数的下标 i ，如果有多个 i 满足条件，请返回最大的 i
33 |         if (diff <= minDiff) {
34 |             minDiff = diff;
35 |             idx = i;
36 |         }
37 |     }
38 |
39 |     return idx;
40 | }
41 |
42 | int main() {
43 |     string s;
44 |     cin >> s;
45 |
46 |     // 输入串最后一个逗号的位置
47 |     int index = s.rfind(',');
48 |
49 |     stringstream ss(s.substr(1, index - 1));
50 |     string token;
51 |
52 |     vector<int> x;
53 |     while (getline(ss, token, ',')) {
54 |         x.emplace_back(stoi(token));
55 |     }
56 |
57 |     int k = stoi(s.substr(index + 1));
58 |
59 |     cout << solution(x, k);
60 |
61 |     return 0;
62 | }

```

